

技术综述教学笔记

Kimi Linear、Sparse Attention 与下一代模型架构

Zhang Xiaojun 对话杨松琳

基于 Zhang Xiaojun Podcast 公开视频、GPU 转写稿、章节结构与自制概念图整理

2025-11-03



视频标题: 119. Kimi Linear、Minimax M2? 和杨松琳考古算法变种史，并预演未来架构改进方案

视频频道: Zhang Xiaojun Podcast

视频时长: 01:43:26

视频链接: <https://www.youtube.com/watch?v=858HR43pegk>

素材说明: YouTube 无可利用字幕；正文基于 faster-whisper large-v3 CUDA 转写、视频章节、封面和自制教学图整理。固定访谈画面不在正文重复使用。

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 导读: 为什么 2025 年又开始“雕 Attention”	3
1.1 本章小结	3
2 Attention 的问题地图: 长 CoT、KV Cache 与 Decoding	3
2.1 Full Attention 的粗略复杂度	4
2.2 KV Cache 为什么重要	4
2.3 本章小结	5
3 Linear Attention: 把平方矩阵改写成线性递推	5
3.1 它在 Transformer 里的位置	6
3.2 本章小结	6
4 Kimi Linear 与 KDA: 更细粒度的记忆更新	7
4.1 Scaling Ladder: 小规模通关再放大	7
4.2 三比一的混合比例	8
4.3 本章小结	9
5 三条 Attention 路线: Kimi、DeepSeek、MiniMax	9
5.1 Kimi vs DeepSeek	10
5.2 MiniMax M2 为什么回到 Full Attention	10
5.3 本章小结	11
6 从 MoE 到 Attention: 架构创新为什么重新重要	11
6.1 FLOPs、Loss 与架构	12
6.2 为什么数据撞墙会让算法创新更重要	12
6.3 本章小结	13
7 算法考古: 旧论文如何在新硬件上重现光芒	13
7.1 为什么旧工作会被淹没	14
7.2 研究品味: 知道什么值得做	14
7.3 开源实现也是研究贡献	14
7.4 本章小结	15
8 硬件亲和: 算法必须能并行、能矩阵乘、能 scale	15

8.1	Chunkwise Algorithm 与并行	15
8.2	DeepSeek 的硬件协同	16
8.3	本章小结	17
9	未来路线: Linear 与 Sparse 会不会合流	17
9.1	为什么组合路线有吸引力	17
9.2	什么叫“选得准”	17
9.3	本章小结	18
10	年轻研究者如何进入架构研究	18
10.1	为什么算力是入场券	18
10.2	本章小结	19
11	术语消化: 本期关键词索引	19
11.1	本章小结	19
12	总结与延伸	20
12.1	核心结论	20
12.2	开放问题	20
12.3	拓展阅读	20

1 导读：为什么 2025 年又开始“雕 Attention”

本节先建立整期的技术背景。杨松琳讨论的是大模型基础架构里最核心的模块之一：Attention。过去几年，Transformer 的 FFN 部分已经被 MoE (Mixture of Experts, 混合专家) 大幅改造，DeepSeek 等模型让 MoE 成为共识；下一块可能被“雕”的，就是 Attention。原因很直接：长上下文、长 CoT、Agentic AI 和长文本 decoding，把 Full Attention 的计算与 KV cache 压力推到了前台。

本期要回答四个问题。第一，Linear Attention 的“linear”到底线性在哪里。第二，Kimi Linear 的 KDA 模块为什么重要。第三，Kimi 的混合线性注意力、DeepSeek 的稀疏注意力、MiniMax M2 回到 Full Attention，分别意味着什么取舍。第四，为什么架构研究不能只讲数学优雅，还必须和硬件亲和、并行算法、矩阵乘和 kernel 优化协同。

本期核心命题

当数据增长变慢、长上下文需求上升、推理成本变重时，架构创新会重新变得关键。Attention 的新路线不是为了替换 Transformer 这个名字，而是为了在相同 FLOPs 下获得更低 loss、更低 KV cache 压力和更好的长序列 decoding 效率。

视觉策略说明

本视频是固定访谈画面，没有 slides 或论文投屏。正文不重复插入人物帧；所有图均为自制概念图，用来解释注意力复杂度、KDA、三条 Attention 路线、Scaling Ladder、算法考古和硬件亲和。

1.1 本章小结

EP119 是架构综述而不是普通访谈。它把 2025 年模型公司在 Attention 上的不同押注放在同一张地图中：Kimi 走 Linear/Hybrid，DeepSeek 走 Sparse，MiniMax M2 暂时回到 Full Attention。

2 Attention 的问题地图：长 CoT、KV Cache 与 Decoding

上一章说为什么要重新研究 Attention，本章先把问题拆开。Full Attention 的好处是表达力强、训练稳定、经验成熟；坏处是在长序列下计算和显存都贵。长 CoT 和 Agentic AI 会生成几万 token 的推理链，decoding 逐 token 生成，KV cache 越来越大，推理成本成为真实瓶颈。

Attention 问题地图

长上下文推理把 KV cache、decoding 和全局注意力推到瓶颈



读图：从左到右看依赖关系；正文负责展开细节，图只保留骨架。

图 1: Attention 问题地图：长上下文推理把 KV cache、decoding 和全局注意力推到瓶颈。自制概念图，依据 00:12:20–00:14:39 会谈内容整理。

读图：长上下文不是只增加输入长度

长 CoT 会让模型在生成阶段持续产出 token。Full Attention 每一步要和历史上下文交互，KV cache 要保存历史 key/value，decoding 成本随上下文变长而增加。新 Attention 的目标，就是降低这些成本而不明显掉性能。

2.1 Full Attention 的粗略复杂度

标准自回归 Softmax Attention 中，给定序列长度 L ，模型会构造 QK^T 得到 $L \times L$ 的注意力分数矩阵，再做 causal mask、softmax 和乘以 V 。因此训练或 prefill 阶段的注意力计算通常有平方复杂度：

$$\text{cost}_{\text{full}} = O(L^2).$$

其中， L 表示序列长度； Q, K, V 分别是 query、key、value 表示； $O(L^2)$ 表示成本随序列长度平方增长。

复杂度不是唯一指标

Full Attention 理论复杂度高，但它由大矩阵乘法构成，硬件亲和很好。很多线性或稀疏算法虽然理论复杂度更低，但如果并行性差、kernel 难写、内存访问不友好，实际速度未必更好。

2.2 KV Cache 为什么重要

KV cache 是推理时保存历史 key/value 的缓存。它让模型在生成新 token 时不必重复计算全部历史表示，但代价是显存随层数、序列长度、batch size 和隐藏维度增长。混合线性注意力路线的一

个优点，是大量层可以像 RNN 递推状态一样工作，从而减少 KV cache；稀疏注意力路线则更多通过只关注 Top-K 或窗口 token 来减少每步计算。

KV cache 的粗略账本

如果一个模型有 N 层，每层保存 key 和 value，序列长度为 L ，batch size 为 B ，每个 token 的 KV 表示维度为 d ，那么 KV cache 的量级可写成：

$$\text{KV cache} \propto 2 \times N \times B \times L \times d.$$

其中，前面的 2 来自 key 和 value 两份缓存。这个式子说明：只要上下文长度 L 或 batch size B 增长，显存压力就会线性增加；如果每一层都保留 Full Attention 的 KV cache，长上下文推理会非常贵。

为什么省 KV cache 会提高吞吐

KV cache 变小以后，同样显存里可以放下更大的 batch size；更大的 batch size 又能提高服务端吞吐。因此 Kimi 这类 hybrid linear 路线不只是减少显存，还会影响推理服务的经济性。

2.3 本章小结

Attention 的瓶颈不是单一 FLOPs 问题，而是长序列下的计算、显存、batch size、decoding 延迟和硬件效率共同作用。理解这些约束，才能理解 Kimi、DeepSeek、MiniMax 为什么选择不同路线。

3 Linear Attention: 把平方矩阵改写成线性递推

本章解释 Linear Attention 的基本直觉。所谓 linear，不是模型变成线性函数，而是复杂度相对序列长度接近线性。典型做法是去掉或改写 Softmax，通过代数变换把注意力写成类似 RNN 的 recurrence，每一步维护一个状态，整体处理长度 L 的序列时成本接近 $O(L)$ 。

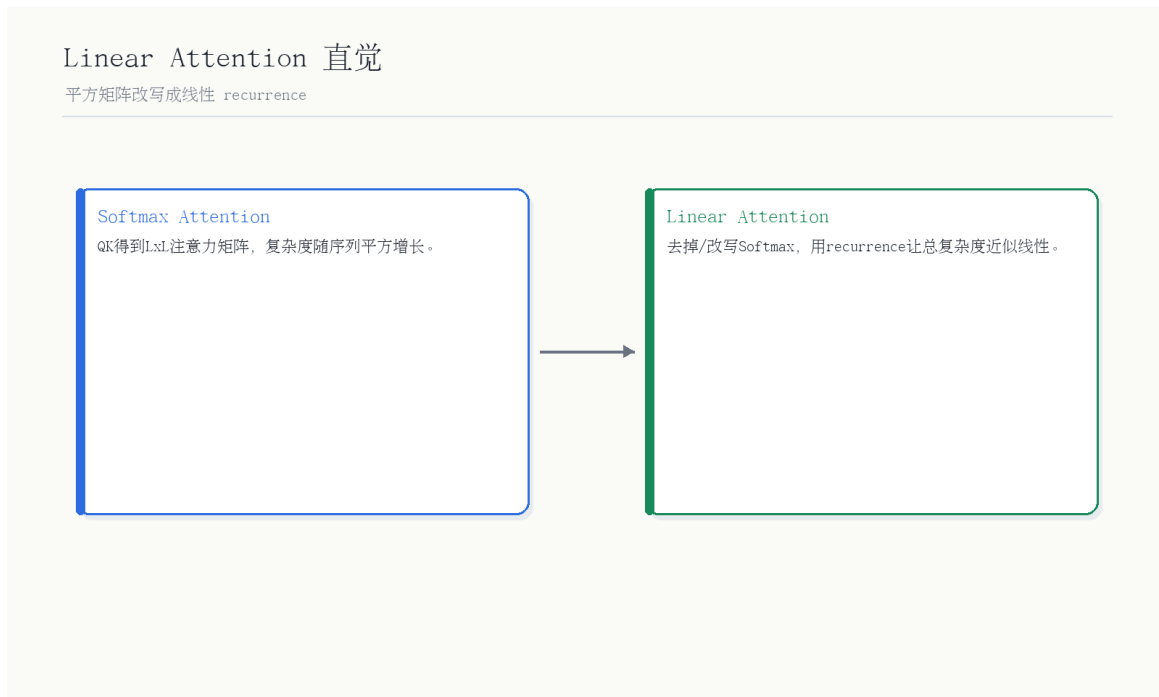


图 2: Linear Attention 直觉：平方矩阵改写成线性 recurrence。自制概念图，依据 00:07:04–00:11:19 对谈内容整理。

读图：Linear Attention 的关键不是少写一层

Softmax Attention 显式构造 $L \times L$ 矩阵；Linear Attention 试图把历史信息压入递推状态，让每一步生成只更新状态。这样能降低复杂度和 KV cache 压力，但表达能力和训练稳定性要重新设计。

3.1 它在 Transformer 里的位置

Transformer 主要由 Attention 和 FFN 反复堆叠构成。近几年，FFN 被 MoE 改造；Linear Attention 则是在 Attention 模块上动刀。它属于 pre-training 架构研究的一部分，和 optimizer、pre-training data、post-training 等不同。现在更常见的是 hybrid 架构：部分层保留 Full Attention，部分层换成 Linear Attention。

Hybrid 架构的动机

纯 Linear Attention 可能表达力不足；纯 Full Attention 太贵。Hybrid 架构用少量全局层兜底表达能力，用大量线性层降低成本，是当前更实际的折中。

3.2 本章小结

Linear Attention 的核心是复杂度和状态表示的重写。它能降低长上下文推理成本，但必须解决表达能力、训练稳定、硬件并行和大规模验证问题。

4 Kimi Linear 与 KDA: 更细粒度的记忆更新

本章进入 Kimi Linear 的核心模块 KDA，即 Kimi Delta Attention。节目中提到，KDA 基于 Gated DeltaNet 等早期工作，并把粗粒度 decay 改为更细粒度 decay：过去一个 attention head 下不同维度共享一个衰减率，现在每个维度可以有自己的衰减率。直觉是让有限 hidden state 更充分地表达不同时间尺度的记忆。



图 3: Kimi Linear / KDA: Kimi Delta Attention 用更细粒度 decay 改善记忆利用。自制概念图，依据 00:14:39–00:18:56 对话内容整理。

读图：KDA 是旧思想的新组合

Gated DeltaNet 提供基础，粗粒度 decay 保证效率，细粒度 decay 增强表达，KDA 重新组织记忆更新，最后进入 hybrid 架构。很多新架构不是凭空发明，而是把旧机制在新硬件和新规模下重新组合。

4.1 Scaling Ladder: 小规模通关再放大

本节解释 Kimi 如何降低架构试错风险。线性注意力不是写完公式就能直接上大模型，必须经过逐级规模验证。

Kimi 内部使用 Scaling Ladder: 一个模块先在小规模下和 Full Attention 对比，表现足够好才进入更大规模继续 scale。这个过程像通关，避免直接在最大规模上烧算力试错。KDA 的设计也是经过多轮混合方式筛选和规模放大后，逐步成为候选。



图 4: Scaling Ladder: 小规模通关后再上更大规模, 与 Full Attention 对比。自制概念图, 依据 00:18:56–00:20:20 对谈内容整理。

读图：架构不是一次性拍脑袋

小规模筛选降低成本，指标达标后进入更大规模，再与 Full Attention 基线比较。只有成本、性能和 scaling 表现都过关，才可能成为上线候选。

4.2 三比一的混合比例

接下来讨论 hybrid 架构里的比例问题。保留多少全局层、替换多少线性层，本质上是在表达能力和效率之间找一个可扩展的工程点。

访谈提到，Kimi Linear 每三层 KDA 插入一层全注意力层，三比一比例逐渐成为一种共识。这个比例的本质是用 Full Attention 维护全局表达能力，用 KDA 降低大部分层的推理成本。它不是数学定理，而是工程实验中形成的经验折中。

KDA 机制消化

机制	作用	为什么重要
Delta Rule	用递推方式更新记忆状态	让 attention 具有类似 RNN 的状态更新能力。
Gating	控制信息保留和遗忘	让模型选择哪些历史信息应该留下。
Decay	记忆衰减率	不同信息可以有不同时间尺度。
Fine-grained decay	每个维度有独立衰减	更充分利用有限 hidden state。
Hybrid 全局层	定期插入 Full Attention	兜底全局表达和长程交互。

KDA 不是“把 Softmax 删掉”这么简单

如果只去掉 Softmax，模型可能表达力不足。KDA 的价值在于重新设计状态更新、门控和衰减，让线性层尽量保留有用历史信息；Full Attention 层则负责补全全局交互。

4.3 本章小结

Kimi Linear 的关键不是“全部换成线性注意力”，而是在 hybrid 架构中用 KDA 提供高效记忆更新，再用少量全局层保证下限。Scaling Ladder 则是让这个设计能逐步走向大规模训练的机制。

5 三条 Attention 路线：Kimi、DeepSeek、MiniMax

前面讲 Kimi，本章把三条路线放在一起比较。Kimi 走 Linear/Hybrid，DeepSeek 走 Sparse Attention，MiniMax M2 从 M1 的 Linear/Hybrid 回到 Full Attention。这三个选择都在回答同一个问题：怎样在长上下文 decoding 中降低成本，同时不牺牲太多表达能力和泛化能力。



图 5: 三条 Attention 路线: Kimi、DeepSeek、MiniMax 的不同押注。自制概念图, 依据 00:20:20–00:27:00 对谈内容整理。

读图: 三条路线解决同一个痛点

Linear/Hybrid 节省 KV cache, Sparse 减少每步关注 token, Full Attention 表达力强但贵。Kimi 选择混合线性, DeepSeek 更偏动态稀疏, MiniMax M2 回到全局注意力, 说明不同团队对成本、稳定性和任务表现的权衡不同。

5.1 Kimi vs DeepSeek

Kimi 的 hybrid 路线保留部分全局注意力层, 大部分层换成线性注意力, 因此能显著减少 KV cache, 同时提高 decoding 效率。DeepSeek Sparse Attention 则通过 indexer 选择 Top-K token, 减少每步 token 参与计算, 但不一定减少每层 KV cache 存储。二者都面向长上下文效率, 但优化点不同。

不要把 Linear 和 Sparse 当作谁绝对更先进

Linear/Hybrid、Sparse 和 Full Attention 各有场景。表达能力、长程检索、KV cache、batch size、硬件实现、训练稳定性都会影响最终选择。模型公司回退到 Full Attention 不一定说明 Linear 失败, 也可能说明当前任务和稳定性更需要全局表达。

5.2 MiniMax M2 为什么回到 Full Attention

节目中提到, MiniMax M1 是大规模混合线性注意力的先驱之一, 但 M2 又回到 Full Attention。一个可能原因是 Linear Attention 在一些 multi-hop reasoning 或复杂检索任务上表现不够稳; 另一

个原因是 Full Attention 的工程确定性、表达下限和训练经验仍然更成熟。架构选择不是只看推理速度，而是看综合表现。

三路线对照表			
路线	优点	代价	适合观察的问题
Full Attention	表达力强、经验成熟	KV cache 和长上下文成本高	多跳推理、复杂检索是否更稳。
Linear/Hybrid	省 KV cache、decoding 友好	需要弥补表达能力和长程交互	长 CoT、Agent 推理吞吐。
Sparse Attention	每步少看 token、硬件可协同	选 token 的 indexer 很关键	能否准确选中有用上下文。

架构选择是多目标优化

模型公司不是在选“最优算法”，而是在性能、成本、稳定性、硬件实现、团队经验和产品场景之间做多目标优化。不同公司回到不同路线，是正常现象。

5.3 本章小结

三条路线的分歧说明 Attention 还没有收敛。Kimi 更愿意用 hybrid 省成本，DeepSeek 更追求硬件协同下的动态稀疏，MiniMax 选择回到更稳的 Full Attention。未来可能出现组合方案。

6 从 MoE 到 Attention: 架构创新为什么重新重要

本章把 Attention 放进更大的架构史。Transformer 包含 Attention、FFN 和位置编码。FFN 已经被 MoE 改造; 位置编码从绝对位置走向相对位置、RoPE 等; Attention 则在 Full、Linear、Sparse、Sliding Window 等路线中继续演化。访谈中的核心判断是: MoE 是近几年最大架构突破，下一块可能就是 Attention。



图 6: 从 MoE 到 Attention: FFN 已经被雕成 MoE, Attention 成为下一块可雕区域。自制概念图, 依据 00:58:08–01:02:48 对话内容整理。

读图：Transformer 不是铁板一块

Transformer 是 Attention + FFN 的堆叠。FFN 变成 MoE 后，算力使用方式发生变化；Attention 仍是长上下文瓶颈，因此 Linear/Sparse 等路线试图继续“雕”架构。

6.1 FLOPs、Loss 与架构

本节把架构选择转成可比较的目标函数。模型公司不是为了新奇而改架构，而是为了在同样计算预算下得到更好的训练和推理结果。

架构创新背后的朴素目标是：给定相同 FLOPs，取得更低 loss。FLOPs 是 floating point operations，即浮点运算量。好的架构不是简单减少计算，而是在相同预算下更有效地表达函数、更好地使用数据、更高效地适配硬件。

架构创新的目标函数

在相同数据和算力约束下，架构创新要么降低训练/推理成本，要么提高表达能力，要么改善硬件效率；最理想的是三者同时改善。

6.2 为什么数据撞墙会让算法创新更重要

接下来把算法放回产业约束中。只要数据和算力都还能无脑增长，架构创新的重要性就容易被掩盖；当两者都变贵时，算法效率会重新进入中心。

节目开头提到，数据、算法、算力是驱动人工智能的三驾马车。当数据增长变慢、算力受限或昂

贵时，算法和架构的价值会重新上升。中国公司算力相对美国更受限，反而更有动力在 MoE、Sparse Attention、Linear Attention、NoPE/RoPE 等方向上做“省算力但不掉性能”的创新。

三驾马车的约束变化

变量	过去的主要做法	现在的压力
数据	继续扩大互联网/代码/多模态数据	高质量公域数据增速放缓。
算力	堆 GPU、扩大训练规模	成本、供给和能源约束加重。
算法	沿用 Transformer 主干微调	需要在相同 FLOPs 下更有效。

6.3 本章小结

Attention 之所以重新成为焦点，是因为长上下文和推理成本把全局注意力推到瓶颈位置。MoE 改造了 FFN，下一阶段可能是 Linear/Sparse/Hybrid Attention 改造注意力层。

7 算法考古：旧论文如何在新硬件上重现光芒

杨松琳强调“考古”：读旧论文、理解旧机制，然后在新问题和新硬件下重新组合。许多机制早在 2016、2020、2021 年就出现过，但当时缺少合适的硬件、并行算法、开源实现或大规模验证，所以被埋在文献海里。DataNet、Delta Rule、Gated Linear Attention 等都属于这种重新被激活的线索。



图 7: 算法考古路线：旧论文、旧机制和新硬件重新组合。自制概念图，依据 01:23:12–01:29:50 对谈内容整理。

读图：考古不是怀旧，而是寻找可复用工具

Transformer、Sparse、DeltaNet、Gated Linear、KDA 之间不是直线进步，而是旧机制在新上下文中被重新评估。很多想法不是第一次出现，而是第一次具备 scale up 条件。

7.1 为什么旧工作会被埋没

本节解释为什么“考古”不是装饰。很多旧论文不是错了，而是在当时缺少让它们变成主流的工程条件。

旧工作被埋没可能有三种原因。第一，配套算法和 kernel 不成熟，别人难以复现或 scale。第二，当时硬件和任务不需要它，价值没有显现。第三，开源代码不好用，社区无法 follow up。杨松琳强调把代码做好，让技术能流传下去，这其实是架构研究影响力的重要组成部分。

小规模有效不代表大规模有效

架构变体很多，许多算法在小规模任务上 work，但到大规模 pre-training 会失效。Scaling Ladder、硬件实现和开源复现，是避免小规模幻觉的重要机制。

7.2 研究品味：知道什么值得做

杨松琳说，真正大的 challenge 不是技术问题本身，而是不知道要做什么。她的做法是先把领域里值得看的论文都读一遍，形成问题地图，再判断哪个旧机制在今天有价值。比如 DataNet 早在 2021 年就有，但缺硬件效率保证；当任务需求和并行算法成熟后，它就可能重新成为可用组件。

算法研究的品味

好的架构研究不是随机组合模块，而是知道历史上有哪些工具、它们为什么没成功、今天哪些约束变了，以及如何用新算法和硬件把旧思想 scale up。

7.3 开源实现也是研究贡献

Flash Linear Attention 之所以重要，不只是因为论文，而是因为它让研究者和业界能方便试线性注意力。如果一个想法没有可用实现，后续研究很难 follow；如果实现好用，技术路线更容易流传和被验证。

从论文到路线的三步

第一步是 idea：数学上或机制上 make sense。第二步是 implementation：能否写成可用、可复现、可优化的代码。第三步是 scaling：能否在更大模型、更真实任务和更硬件友好的条件下成立。

7.4 本章小结

算法考古提供了一种研究方法：不要只追最新 buzzword，要读旧论文、理解机制、判断哪些想法在今天的硬件和任务下可以重新发光。

8 硬件亲和：算法必须能并行、能矩阵乘、能 scale

前面讲了算法考古，本章进入能否落地的最后一道门槛：硬件。一个想法如果不能并行、不能写成高效 kernel、不能贴近现代 GPU 的矩阵乘能力，就很难进入 frontier 模型。

最后一条技术主线是硬件亲和。杨松琳认为，一个算法不仅要在 machine learning 上 make sense，还要能并行、能 scale、能在硬件上高效实现。Transformer 当年能成功，不只是因为表达能力强，也因为 attention 和 FFN 都有大量矩阵乘，硬件亲和优于 RNN/LSTM。



图 8: 硬件亲和算法: 数学 make sense, 还要能并行、能矩阵乘、能 scale。自制概念图, 依据 01:29:50–01:42:23 对谈内容整理。

读图：现代架构研究是算法和系统共同设计

数学合理只是第一步；还要有并行算法，如 chunkwise；要能利用矩阵乘；要有 kernel 实现；最后还要在大模型规模上 scale。任何一环缺失，算法都难以进入 frontier 模型。

8.1 Chunkwise Algorithm 与并行

本节把硬件亲和具体化为并行算法。线性注意力常有递推形式，递推本身不适合 GPU 大规模并行，所以必须把它改写成可分块并行的形式。

Linear Attention 常常有 recurrent 形式，但 recurrence 天然难并行。要让它在现代 GPU 上高效训练，就需要 chunkwise algorithm 等并行算法，把长序列切成块，既保留递推结构，又能利用并行计算。这个角色类似 FlashAttention 对 Softmax Attention 的作用：不是改变模型目标，而是让算法在硬件上跑得起来。

并行算法的意义

没有高效并行算法，理论上很美的 attention 变体也无法进入大规模训练。架构创新和系统优化不是先后关系，而是共同决定一个想法能否落地。

8.2 DeepSeek 的硬件协同

接下来用 DeepSeek 作为硬件协同样本。它的价值不只是选择 sparse，而是让 sparse 的实现形式贴近 FP8、矩阵乘和去 softmax 的硬件路径。

访谈中认为 DeepSeek 非常重视硬件和算法协同。DeepSeek Sparse Attention 的 indexer 可以用 FP8 计算 attention score / logit，去掉昂贵的 softmax/exp 操作，留下大量矩阵乘和 Top-K 选择。这使得它更贴近硬件原则，也更容易和 infra 团队协同。



图 9: DeepSeek 的硬件协同：稀疏选择、FP8 和矩阵乘共同服务效率。自制概念图，依据 01:29:50–01:42:23 对话内容整理。

读图：硬件协同不是只写 kernel

算法侧通过 Sparse / Indexer / Top-K 减少计算；硬件侧用 FP8、矩阵乘、去掉昂贵 softmax/-exp 等方式提高效率。真正的 co-design 是算法目标和硬件原则同时考虑。

8.3 本章小结

未来架构研究必须同时满足三件事：机器学习上合理，数学上可解释，硬件上能并行和矩阵乘。否则再漂亮的算法也很难进入 frontier 模型。

9 未来路线：Linear 与 Sparse 会不会合流

本章把前面几条路线合起来看。节目中提到一个未来理想方案：把 Linear Attention 和 Sparse Attention 结合。直觉上，Linear/Hybrid 负责降低 KV cache 和大部分层的递推成本，Sparse 负责在需要全局检索时选中少数关键 token。理论上，如果 sparse 选得准，它可以替代一部分 Full Attention 层。

9.1 为什么组合路线有吸引力

本节解释为什么未来很可能不是单一路线胜出。Linear、Sparse、Full 各自解决不同瓶颈，真正的工程系统往往会把它们组合起来。

Linear Attention 擅长省状态和降低大多数 token 的成本，但可能牺牲一些复杂全局交互；Sparse Attention 保留“看历史中重要位置”的能力，但需要一个足够可靠的 indexer。组合路线希望两者互补：大部分时间用 cheap recurrence，必要时用 sparse retrieval 补全长程信息。

组合路线的三种可能

组合方式	机制	主要风险
Linear + Full	多数线性层，少数全局层	全局层仍可能限制 context window。
Sparse + Full	多数稀疏层，少数全局层	KV cache 仍大, indexer 要可靠。
Linear + Sparse	线性状态加稀疏检索	训练稳定性和实现复杂度更高。

9.2 什么叫“选得准”

Sparse Attention 的核心不是少看 token，而是少看时还要看对。对于多跳推理、长文档检索、代码依赖和 agent 轨迹，关键信息可能出现在很远的位置。Indexer 如果选错，计算省了，但模型答错；如果选太多，又失去稀疏优势。

Sparse 的失败模式

稀疏注意力最危险的失败不是速度慢，而是静默漏掉关键 token。长上下文任务里，漏掉一个早期条件或远处约束，可能导致后续推理全错。

9.3 本章小结

未来 Attention 可能不是 Linear、Sparse、Full 三选一，而是组合。真正的难点在于如何让组合路线同时保持表达力、低成本、硬件亲和和训练稳定性。

10 年轻研究者如何进入架构研究

前面已经把 Attention 的技术路线和硬件约束讲完，本章把讨论落回研究方法。架构研究不是单纯读论文，也不是只会写公式；它需要接近真实模型、真实算力和真实硬件约束。对于年轻研究者来说，最重要的问题是怎样进入真正能验证 scale 的环境。

节目结尾给年轻研究者的建议很直接：找一个 lab 实习，跟上 frontier，因为做架构必须有算力。这个建议听起来朴素，但背后有很现实的原因：架构研究很难只靠小规模实验判断，小模型上 work 的想法到大模型未必 work。



图 10: 进入架构研究的路径：考古、算力、frontier lab 和硬件意识缺一不可。自制概念图，依据 01:42:23–01:43:26 对话内容整理。

读图：架构研究需要靠近真实约束

读旧论文建立地图，找到有用问题，获得足够算力验证 scale，进入 frontier lab 接近真实训练环境，再用开源实现让技术流传。这是一条从理解到验证再到影响的路径。

10.1 为什么算力是入场券

架构研究的许多问题只有到一定规模才暴露。表达力、训练稳定、KV cache、decoding、kernel、batch size、硬件吞吐，这些都不是单机小实验能完全验证的。没有算力，就很难判断一个架构

是未来路线，还是小规模幻觉。

10.2 本章小结

进入架构研究，既要有算法品味，也要有工程和硬件意识。年轻研究者不能只读最新论文，也不能只做玩具实验；最重要的是靠近真实 frontier 约束。

11 术语消化：本期关键词索引

术语	一句话解释	在本期中的作用
Full Attention	标准 Softmax 全局注意力	表达强，但长上下文成本高。
Linear Attention	近似线性复杂度的注意力变体	Kimi/Qwen 等路线重点探索。
Sparse Attention	只选择部分 token 参与注意力	DeepSeek 路线重点探索。
KV Cache	推理时保存历史 key/value 的缓存	长上下文显存瓶颈。
KDA	Kimi Delta Attention	Kimi Linear 的核心线性注意力模块。
Delta Rule	一类递推更新规则	KDA/Gated DeltaNet 的历史线索。
Gated DeltaNet	带门控的 DeltaNet 变体	KDA 的基础之一。
Decay	记忆衰减率	细粒度 decay 改善 hidden state 使用。
Hybrid Attention	Full 与 Linear/Sparse 混合架构	当前实用路线。
Scaling Ladder	逐级放大规模验证架构	Kimi 内部评估机制。
MoE	Mixture of Experts，混合专家	FFN 侧已被证明的架构突破。
FLOPs	浮点运算量	比较计算成本与效率。
NoPE / RoPE	无位置编码 / 旋转位置编码	训练细节与位置建模相关。
Chunkwise Algorithm	分块并行算法	让 recurrence 类算法能并行训练。
Hardware-friendly	硬件亲和	算法能否大规模落地的关键。

11.1 本章小结

本期术语密度很高，但核心关系很清楚：长上下文让 Full Attention 成本上升，Linear/Sparse/Hybrid 都在尝试降低成本；真正有前途的路线必须同时兼顾表达能力、效率和硬件亲和。

12 总结与延伸

12.1 核心结论

1. 2025 年架构创新重新重要，是因为数据增长变慢、长上下文需求上升、推理成本变重。
2. Full Attention 表达力强，但长序列下计算和 KV cache 都贵。
3. Linear Attention 的核心是把平方注意力改写成接近线性的 recurrence 或状态更新。
4. Kimi Linear 的 KDA 用更细粒度 decay 改善有限 hidden state 的记忆利用。
5. Kimi 走 Hybrid Linear，DeepSeek 走 Sparse，MiniMax M2 回到 Full，说明 Attention 还未收敛。
6. FFN 已经被 MoE 改造，Attention 是 Transformer 里下一块可雕的区域。
7. 算法考古能把旧论文中被埋没的机制放到新硬件和新规模下重新验证。
8. 硬件亲和是架构研究底线：要能并行、能矩阵乘、能写 kernel、能 scale。
9. DeepSeek 的优势之一，是算法和 infra/硬件协同意识强。
10. 年轻研究者想做架构，需要接近 frontier lab 和算力环境，否则很难验证 scale。

12.2 开放问题

最后保留开放问题，是因为 Attention 路线还没有收敛。Kimi、DeepSeek、MiniMax 的选择都不是终局，而是不同公司在不同约束下的阶段性押注。

- Kimi Linear 这类 Hybrid Attention 能否在更大规模持续接近 Full Attention 表现？
- Sparse Attention 能否在保留长程检索能力的同时真正替代全局层？
- 三比一的 hybrid 比例会成为长期经验，还是过渡方案？
- Full Attention 是否会因为硬件继续优化而延长寿命？
- MoE 之后，Attention 的突破会不会成为下一轮架构共识？
- 架构研究如何在开源小规模 and 闭源大规模之间建立可靠验证桥梁？

12.3 拓展阅读

- Kimi Linear: An Expressive, Efficient Attention Architecture。
- DeepSeek Sparse Attention / Native Sparse Attention 相关技术报告。
- FlashAttention 系列：理解算法和硬件协同。
- MoE、RoPE、NoPE、Mamba、DeltaNet、Gated Linear Attention 等架构材料。
- Hazy Research 相关博客：高效序列建模和硬件友好架构。

最后的判断

EP119 最值得留下的是一种研究姿势：不要只问“哪个 attention 更火”，而要问它解决哪个瓶颈、牺牲什么表达能力、如何在硬件上并行、能否通过 scaling ladder，以及是否能在真实 frontier 模型里稳定兑现。